



HOW TO ACTUALLY DO

Spec-Driven Development

Layered specifications that reduce ambiguity one stage at a time

AI coding agents do not perform better when you write more. They perform better when each document answers exactly one question.

One enormous document that answers everything, badly

The 200-page specification

- Business context buried beside endpoint definitions
- Nobody reads it end to end — including the agent
- A change to one API forces a rewrite of the whole file
- Retrieval pulls in 190 irrelevant pages
- Review becomes impossible, so review stops happening

What actually helps

- Small documents, each with one job
- Ambiguity removed progressively, stage by stage
- Version-controlled alongside the code
- Retrievable in isolation — the agent reads only what it needs
- Reviewable in one sitting by one person

The question is never PRD or FRD or SRS. It is which question this particular file is answering.

THE PRINCIPLE

Do not write everything down.

Reduce ambiguity progressively.

Each layer takes the output of the one above it and makes one more category of decision explicit. By the time you reach implementation tasks, there is nothing left for the agent to guess — and nothing was written twice.

Five documents, five different questions

DOCUMENT	ANSWERS	AUDIENCE	AI VALUE
PRD	What problem are we solving?	Product, stakeholders	Medium
FRD	What should the system do?	Business analysts, devs	High
SRS	Exactly how should it behave?	Developers, QA, AI agents	Very High
Design / Architecture	How will we build it?	Engineers	Very High
Implementation tasks	What code needs writing?	Developers, AI agents	Very High

Value to an AI agent rises as ambiguity falls. That is the whole mechanism.

Building a house

PRD

Why build the house?

- 3-bedroom house
- Budget ₹80 lakh
- Family of four
- Natural lighting
- Modern style

No construction detail.

FRD

What rooms should it have?

- 3 bedrooms
- 2 bathrooms
- Modular kitchen
- Parking
- Solar panels
- Home office

Still no engineering.

SRS

Exact specifications

- Bedroom dimensions
- Door sizes
- Electrical outlets
- Plumbing requirements
- Concrete strength
- Window positions

What builders need.

Architecture

How it stands up

- RCC frame
- Brick walls
- Electrical routing
- Plumbing layout

Where things go.

Nobody hands a builder the budget conversation. Nobody hands a stakeholder the concrete grade.

What problem are we solving?

PRD

Build an AI recruitment platform.

Users should:

- Create assessments
- Invite candidates
- View reports
- Monitor cheating

What it establishes

The business problem, the users, the goals and the success metrics. Enough for a stakeholder to agree the thing is worth building.

What it deliberately omits

Workflows, screens, endpoints, data models. An agent given only this will invent all four — which is exactly the failure you are trying to avoid.

Useful for reasoning about the business. Not yet useful for writing code.

What should the system do?

FEATURE: CANDIDATE LOGIN

Actor: Candidate

Preconditions:

Assessment invitation exists.

Flow:

1. Open assessment link.
2. Enter OTP.
3. Verify identity.
4. Start assessment.

Exceptions:

OTP expired.

Assessment already submitted.

One feature per file. Authentication does not belong in the same document as the assessment engine.

Now the agent understands function

- Who the actor is
- What must already be true before the flow starts
- The happy path, in order
- The exceptions that must be handled
- Business rules and edge cases for this feature alone
- Acceptance criteria scoped to one capability

Exactly how should it behave?

POST /login

Input:

```
{ email, otp }
```

Validation:

```
OTP valid for 5 minutes.
```

Response 200:

```
{ token, candidate_id }
```

Errors:

```
401 Invalid OTP
```

```
404 Candidate not found
```

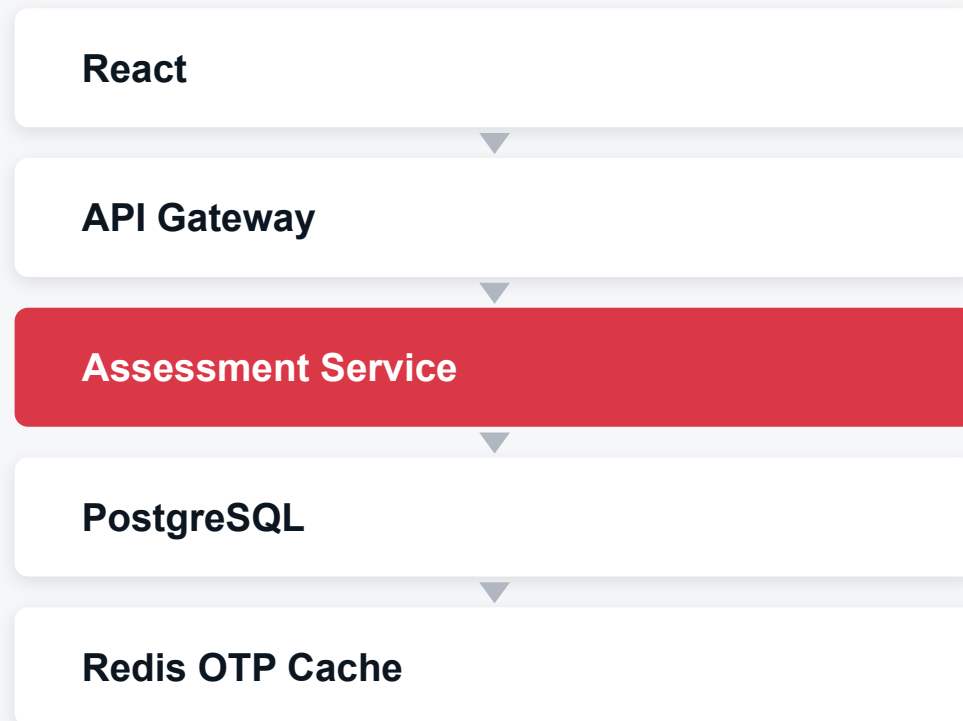
```
429 Too many attempts
```

This is the layer that pays

- Exact request and response shapes
- Validation rules with real values — five minutes, not “soon”
- Every error path, with its status code
- State transitions and their triggers
- Non-functional limits that constrain the implementation

Now the agent can write production-quality code — because there is nothing left to infer.

Where does the code belong?



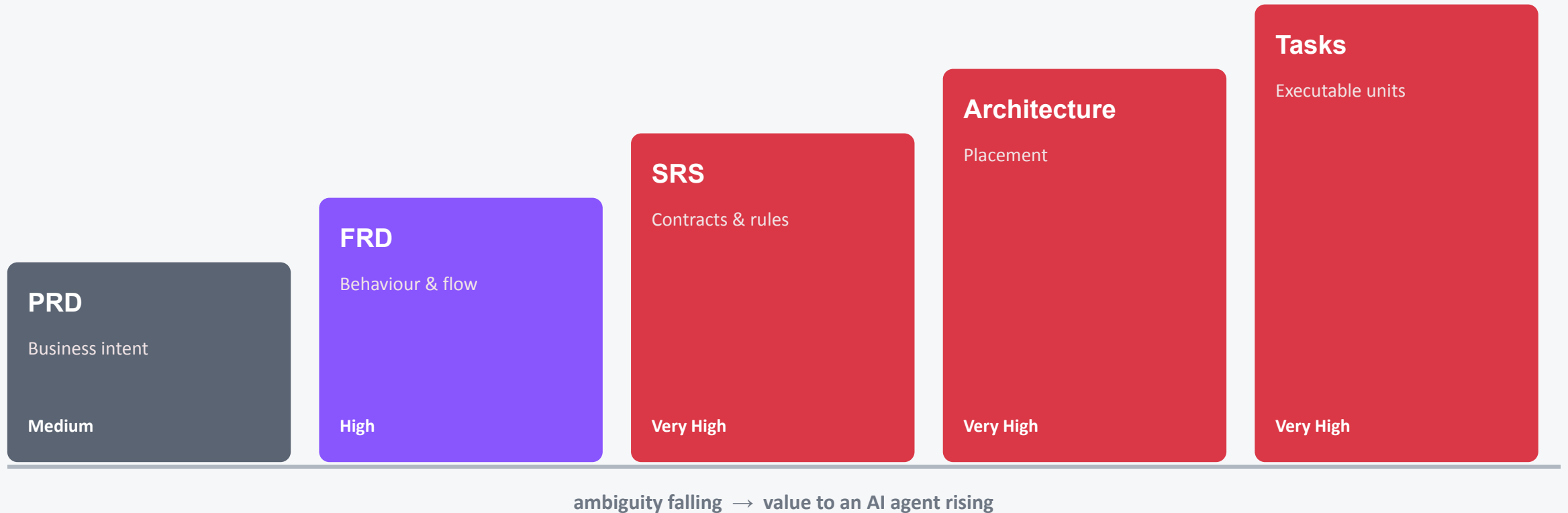
What this buys you

- The agent knows which service a change belongs in
- Boundaries are stated, so they stop being crossed by accident
- Infrastructure choices become reviewable before code exists

Without this layer, an agent will happily put OTP expiry logic in the React component — and it will work, right up until the second client.

WHY THIS ORDER

Every layer removes one category of guesswork



Skip a layer and the agent fills the gap silently. It will not tell you which assumption it made.

From documents to specs

A working repository layout

specs/

product.md

requirements.md

architecture.md

api.md

data-model.md

ux.md

testing.md

tasks/

task-001.md

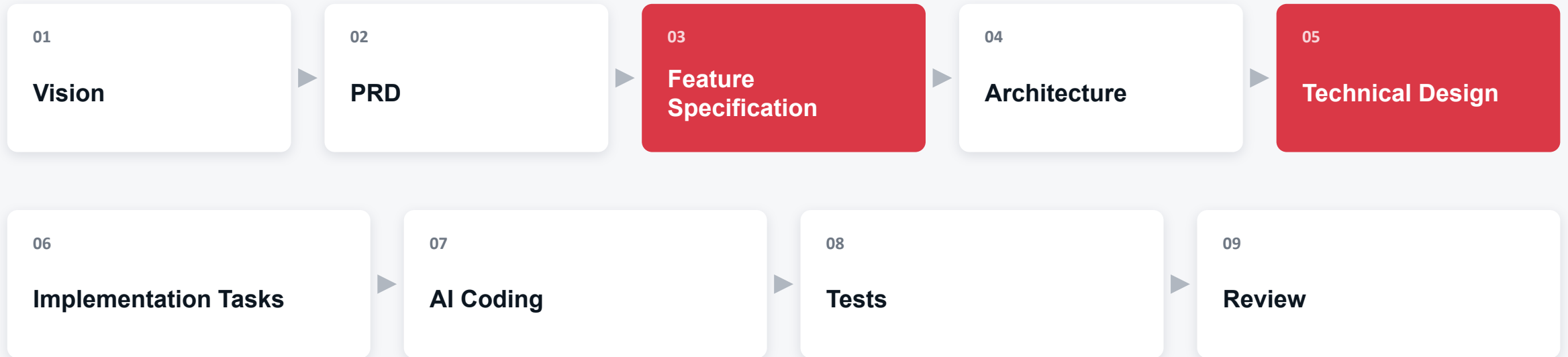
task-002.md

Why teams are moving this way

- Each file is small, focused and independently reviewable
- Version-controlled — a spec change shows up in a diff
- Retrieval pulls the two relevant files, not the whole corpus
- Tasks are numbered units an agent can be handed one at a time
- A stale document is visible, because it stopped changing

The observation is that several AI-native engineering organisations have converged on roughly this shape — treat that as a signal, not a standard.

Nine stages, for AI-first development



Stages 3 and 5 are highlighted for a reason — together they are what an SRS used to be. See the next slide.

It did not disappear. It split.

SRS

The monolithic
IEEE-style document

becomes

Feature Specification

Workflows, business rules, edge cases and acceptance criteria — for one feature at a time.

Technical Design

APIs, data models, validations, state transitions, error handling, non-functional limits.

Splitting it is what makes each half small enough to keep current. A monolith goes stale the week it is signed off.

WORKED EXAMPLE

How this looks for a real product

docs/product/ vision.md roadmap.md personas.md	docs/features/ authentication.md assessment-engine.md proctoring.md jd-parser.md resume-ranking.md	docs/architecture/ system.md frontend.md backend.md ai-services.md database.md	docs/api/ auth.yaml assessments.yaml
docs/data/ entities.md database-schema.md	docs/testing/ acceptance-tests.md integration-tests.md	docs/prompts/ jd-generation.md assessment-generation.md	The point An agent retrieves the two files it needs — never a 200-page document.

Note that prompts are version-controlled alongside the specs. They are source, not configuration.

Do not choose between them. Layer them.

1 PRD

Business problem, users, goals, success metrics.

2 Feature Specifications

Workflows, business rules, edge cases, acceptance criteria — per feature.

3 Technical Specifications

APIs, data models, validations, state transitions, error handling, NFRs.

4 Architecture & plans

Where and how the agent should implement the solution.

Minimise ambiguity at each stage. Keep every document short enough that a person will actually read it.

TAKE THIS AWAY

Three things to change this week

01

One question per file

If a document answers two questions, split it. That single habit produces most of the benefit.

02

Put specs in the repository

Version-controlled beside the code, in a diff you can review. A spec that lives in a shared drive is already stale.

03

Write the value, not the adjective

“OTP valid for 5 minutes” is a specification. “OTP expires quickly” is a wish, and the agent will resolve it for you.

The agent is not short of capability. It is short of decisions you have not yet made.